

Aluno: Pedro Thiago Eloí de Lima

Foram selecionados a priori três algoritmos de ordenação diferentes: bubble sort, selection sort e insertion sort, e também foram implementadas versões melhoradas (uma para cada algoritmo) variando o tamanho das entradas em 10000, 20000 e 50000 números em ordem aleatória.

O objetivo do experimento foi medir e comparar o consumo energético desses diferentes algoritmos conforme o tamanho da entrada aumenta e o algoritmo muda. Nesse contexto, os experimentos foram realizados em um laptop com processador Intel core i5-1235U, com 8GB de RAM em um sistema operacional Fedora Linux 43 (Workstation Edition) e a ferramenta perf foi utilizada para coletar os dados de consumo energético, tempo de execução e tempo de cpu (tempo de usuário + tempo de sistema).

Sobre o funcionamento de cada algoritmo, vamos começar com o bubble sort: ele percorre o vetor n vezes, comparando o elemento no índice atual com o elemento imediatamente próximo a ele, e caso o próximo elemento seja menor que o atual, ele troca os dois valores de posição, ao passo que no final o vetor estará ordenado em ordem crescente. Qual seria uma possível melhoria para esse algoritmo? Bom, ele itera sobre o vetor n vezes, mas isso não significa que o vetor não pode estar completamente ordenado antes da n -ésima iteração. Então, é possível monitorar quando ocorrem trocas de posição entre os valores e quando o algoritmo realiza uma varredura completa e não realiza trocas, significa que o vetor já está ordenado. Isso pode levar a um melhor consumo energético porque pode diminuir a quantidade de varreduras inúteis pelo vetor, encerrando o programa mais cedo e diminuindo seu tempo de execução.

Funcionamento do selection sort: itera sobre o vetor procurando o menor valor, encontra ele e move esse valor para o início do vetor e avança para encontrar o menor elemento que vai substituir o elemento atual do vetor, repetindo esse algoritmo tantas vezes quanto há números no vetor (n vezes para um vetor com n números). Uma melhoria para esse algoritmo pode ser trocar a estrutura de dados por uma heap queue. Como ela funciona? No algoritmo selection sort melhorado, importamos o módulo heapq que implementa os métodos de uma heap queue e criamos uma max heap queue ao armazenar os valores do vetor multiplicados por -1 fazendo com que o maior valor absoluto fique no topo da estrutura e então preenchemos o vetor de trás para frente, garantindo que ele esteja completamente ordenado ao retirar todos os elementos da max heap queue. Essa modificação leva a um menor consumo energético porque a criação da heap queue tem complexidade $O(n \log n)$ que já é menor que a complexidade original do algoritmo, e o acesso aos elementos da heapqueue é constante, porque já se sabe que o valor no topo será menor a cada iteração, por isso preencher o vetor ao contrário funciona perfeitamente para a ordenação.

Por fim, o funcionamento do insertion sort: considera o primeiro elemento do vetor como ordenado, e começa a partir do segundo elemento do vetor. O algoritmo compara se o segundo elemento é menor que o primeiro e troca as posições dos valores caso isso seja verdadeiro. Na prática o vetor tem uma parte ordenada que aumenta a cada iteração porque o algoritmo só avança o valor quando os valores do início até a posição atual estão ordenados, e uma parte desordenada que diminui a cada iteração. É como ordenar cartas de baralho onde a mão esquerda segura as cartas na ordem correta e a mão direita segura as cartas em ordem aleatória, e ao trocar a carta de mão compara a carta nova com cada uma das antigas, abrindo espaço para a inserção dela (por isso insertion sort) no espaço adequado. Uma melhoria possível para esse algoritmo seria garantir que o primeiro elemento do vetor é o menor, fazendo uma busca por ele e posicionando ele no início do vetor antes do laço principal do algoritmo. Isso dispensa a verificação interna se o índice j que verifica a parte ordenada do vetor é maior ou igual a zero porque faz com que a outra condição verificada (se o valor armazenado na variável chave é menor que o valor armazenado na posição j que é anterior à posição atual i) seja suficiente pois se o menor número está garantidamente no início do vetor, não existe o risco do índice j sair dos índices válidos do vetor. Isso pode reduzir o consumo energético do programa porque ao invés de realizar 2 verificações por elemento do vetor, o algoritmo realiza apenas uma. Para um vetor com 50000 números, as operações vão de 100000 verificações por elemento no pior caso para 49999 por elemento no pior caso e isso diminui a quantidade de instruções lançadas para o processador executar.

N	Energia	Tempo Exe	Tempo CPU	Potência Exe	Potência CPU
Bolha					
10000	73,11	5,42	5,43	13,49	13,46
20000	279,01	22,99	22,99	12,15	12,15
50000	1699,95	141,64	141,62	12,00	12,00
Bolha Melhorado					
10000	73,42	6,10	6,11	12,04	12,02
20000	287,86	23,97	24,00	12,01	12,00
50000	1813,98	151,14	151,23	12,00	11,99

Com base nos resultados obtidos dos experimentos, foi observado para o algoritmo bubble sort melhorado houve um ligeiro aumento no consumo energético e no tempo de execução do programa conforme o tamanho da entrada aumentou, entretanto a potência do programa foi reduzida. Isso significa que em um cenário onde os dois algoritmos são executados pelo mesmo tempo, o algoritmo bubble sort melhorado apresentaria um consumo energético menor do que a versão básica. O que mostra que a melhoria feita realmente pode trazer uma redução no consumo energético, principalmente em vetores grandes porém quase ordenados, pois o número de iterações necessários para completar a ordenação seria bem menor do que o tamanho do vetor.

N	Energia	Tempo Exe	Tempo CPU	Potência Exe	Potência CPU
Selection					
10000	38,66	2,86	2,90	13,50	13,31
20000	150,58	12,00	12,18	12,55	12,37
50000	922,41	76,83	77,91	12,01	11,84
Selection Melhorado					
10000	0,15	0,03	0,03	5,13	6,14
20000	0,21	0,03	0,04	6,03	6,35
50000	0,43	0,05	0,05	8,60	9,79

Já no selection sort, a versão melhorada do algoritmo apresentou uma redução significativa no tempo de execução e no consumo energético em comparação com a versão básica. Para a entrada de 10000 números, a potência gerada pelo selection sort melhorado foi menos da metade da potência gerada pela versão inicial. A potência aumentou conforme o tamanho da entrada, mas ela continuou sendo significativamente menor que a versão básica do algoritmo. Para as entradas analisadas, o uso da versão melhorada do algoritmo seria mais energeticamente eficiente em todas elas ao comparar com a versão padrão do algoritmo.

N	Energia	Tempo Exe	Tempo CPU	Potência Exe	Potência CPU
Insertion					
10000	36,85	2,71	2,78	13,62	13,31
20000	136,14	10,83	10,84	12,59	12,58
50000	834,02	69,48	69,53	12,00	12,00
Insertion Melhorado					
10000	31,29	2,33	2,34	13,45	13,37
20000	124,06	9,79	9,81	12,69	12,66
50000	757,43	63,14	63,19	12,00	11,99

A respeito do algoritmo insertion sort, a versão melhorada apresentou uma ligeira redução tanto no tempo de execução quanto no consumo energético, quando comparado com o algoritmo original. Apesar disso, a potência gerada pela versão melhorada do algoritmo foi menor do que a versão base para a entrada de 10000 números, maior para a entrada de 20000 números e igual para ambas as versões para a entrada de 50000 números. Esse resultado contrasta com o que foi observado no bubble sort: se os dois algoritmos do insertion sorte executassem durante o mesmo tempo, a versão melhorada resultaria em um consumo energético maior do que a versão sem alterações, o oposto do que foi observado para o experimento do bubble sort.

Podemos concluir que ao melhorar um algoritmo as alterações que podem levar a um menor consumo energético podem envolver usar estruturas de dados mais adequadas para as tarefas, como observado na versão melhorada do algoritmo selection sort que trocou uma lista comum por uma fila de prioridade e reduziu significativamente o consumo energético dele. Por outro lado, direcionar os esforços para reduzir a quantidade de instruções que o processador executa, como no caso do algoritmo

insertion sort melhorado, não teve um efeito muito significativo na diminuição do consumo energético. Ainda há melhorias que podem beneficiar casos particulares, como no caso do algoritmo bubble sort melhorado, onde em um vetor grande porém quase ordenado o algoritmo não iria iterar sobre o vetor desnecessariamente caso a ordenação seja concluída rapidamente e isso diminuiria o tempo de execução do programa, que é um fator decisivo no consumo energético de um programa.